York University
Lassonde School of Engineering

EECS3311 – Software Design

Market Intelligence Agent

Stage 2 – Implementation Report

Student: Erdem Calikoglu
Student Number: 218188771
Instructor: Song Wang
Date: November 8, 2026

Contents

1 Project overview and implementation status **3**

2 Feature to implementation mapping **4**

2.1 UC-01 Register account 4

2.2 UC-02 Log in 4

2.3 UC-03 Log out 4

2.4 UC-04 Get a market quote 5

2.5 UC-05 Get a security overview 5

2.6 UC-06 Analyze ETF exposure 5

2.7 UC-07 Compare securities 6

2.8 UC-08 Analyze ETF overlap 6

2.9 UC-09 Retrieve financial news 6

2.10 UC-10 Explain a recent move 6

2.11 UC-11 Add a portfolio position 7

2.12 UC-12 Remove a portfolio position 7

2.13 UC-13 View the portfolio 7

2.14 UC-14 Manage the watchlist 8

2.15 UC-15 Analyze portfolio exposure 8

2.16 UC-16 Ask a follow-up 8

2.17 UC-17 Read financial statements 9

2.18 UC-18 Read price history 9

2.19 UC-19 Read sentiment 9

2.20 UC-20 View the home desk 9

3 Implemented design patterns **10**

3.1 Adapter 10

3.2 Composite 10

3.3 Command 10

3.4 Facade 11

3.5 Factory 11

3.6 Also in the code 11

3.7 Diagrams 11

4 Changes and rationale **12**

5 AI tools and collaboration log **14**

6 Development and verification summary **17**

6.1 Weekly commits 17

6.2 What I checked 17

6.3 Defects I hit and fixed 17

6.4 Still open 18

7 Demonstration index **18**

1 Project overview and implementation status

I built a small research desk for one signed-in person. The Swing window and the terminal call the same Java services, and both read the same SQLite file, `data/market-intelligence.db`. You can ask for a quote, a company profile, ETF exposure, news, statements, a price history, or a sentiment score. You can also keep a portfolio and a watchlist. The desk does not place orders and it does not tell you to buy or sell.

Gemini is in two places, and only those two. If you ask why a price moved, the desk first loads the quote and the news, then asks Gemini to explain the move from what those articles say. If you type an ordinary sentence, Gemini may pick one allowed action, and it may write a short reply from the tool result. A plain quote never goes to the model. "Say that more simply" stays on the last reply and does not fetch anything new. Portfolio weights are calculated in Java. I do not send those numbers to Gemini.

The model I ship with is `gemini-3.5-flash-lite`. Flash-Lite uses minimal thinking, because a longer thinking pass was eating the token budget and the note stopped after the price. If `GEMINI_MODEL` is set to `gemini-3.8-flash` or `gemini-2.5-flash`, unset it. The first one thought too long. The second one returned HTTP 404 on a new key.

The box at the bottom of the window is a message field. Enter sends, Shift+Enter adds a line, and the reply stays in the conversation. The terminal takes the same words. A leading slash still works, but you do not need it.

What works today, when the matching key is set:

- Register, sign in, and sign out. The create-account screen and the sign-on screen are separate.
- Quotes, overviews, comparison, news, ETF sector weights, overlap, portfolio edits, and a watchlist.
- Country exposure. Finnhub's published report if the plan allows it. Otherwise the headquarters country of at most 25 holdings.
- Portfolio exposure as price times quantity, with currencies kept apart.
- A move note that uses the articles, then the quote and the sources underneath.
- Statements, daily bars with RSI, and one sentiment score.
- A home screen with portfolio value, the day move, one row per position, and a sliding tape.

What I am not going to dress up:

- The demonstration videos are not public yet. Section 7 is the shot list.
- On the free Finnhub plan, country weights are headquarters of the holdings I could look up. They are not the fund's factsheet and they are not revenue. Anything unknown stays unknown.
- Finnhub's free quotes are US listings. `XEQT.TO` stays unpriced, and the screen says so.
- Alpha Vantage company overview is a paid endpoint. I use Alpha Vantage for ETF holdings.
- If a live call fails, the router does not try the next vendor.
- The model cannot add or remove a position.
- A sentence that tries to override the instructions does not run a tool.

2 Feature to implementation mapping

Stage 1 had UC-01 through UC-16. I kept all of them. UC-17, UC-18, and UC-19 are the statement, history, and sentiment commands I added once those free APIs were wired in. UC-20 is the home screen. Login, logout, and registration work, but I am not counting them toward the ten meaningful features.

For each one I say how you get from the sentence to the result, whether Gemini is involved, and where the clip will go. The video cell is empty until the recordings are public.

2.1 UC-01 Register account

Stage 1. UC-01. Implemented.

I open the create-account screen, which asks me to type the password twice. `AuthService.register` checks the length, `BCryptPasswordHasher` hashes the password, and `SQLiteUserRepository.save` stores the row. The window then sends me back to sign-on. I am not logged in yet. A taken username or a short password is rejected, and the password never lands in the status line. This does not call Gemini. **Classes.** `AuthenticationPanel` collects the form. `SessionWorkspace.register` is what the screen calls. `AuthService` owns the rules.

Methods. `AuthService.register`, `PasswordHasher.hash`, `UserRepository.save`.

Patterns. Strategy for the hasher, Repository for the user row.

Video. Not public. Clip in Section 7.

2.2 UC-02 Log in

Stage 1. UC-02. Implemented.

Sign-on does not ask me to confirm the password. `AuthService.authenticate` loads the user and checks the hash. `SessionManager.startSession` starts the session. A wrong password stays on that screen and does not say which field failed. Deterministic. **Classes.** `AuthenticationPanel` or `TerminalUI`, `AuthService`, `SessionManager`.

Methods. `authenticate`, `PasswordHasher.verify`, `startSession`.

Patterns. Strategy, Repository.

Video. Not public.

2.3 UC-03 Log out

Stage 1. UC-03. Implemented.

Logout clears the in-memory session. The next command asks me to sign in again. The old conversation rows stay in SQLite under that user. Deterministic. **Classes.** `SessionWorkspace.logout`, `SessionManager.endSession`.

Methods. `endSession`.

Patterns. Facade, through `SessionWorkspace`.

Video. Not public.

2.4 UC-04 Get a market quote

Stage 1. UC-04. Modified. I took the model off this path.

I type `quote NVDA`. `ResearchGateway` checks that I am still the signed-in user. `MarketDataTool` checks the symbol and asks `ProviderRouter` for a quote. Finnhub answers if that key is set. Twelve Data, then Alpha Vantage, are used only when an earlier adapter was not registered. A failed call does not hop to the next one. The screen shows price, previous close, and percent change, and NVDA is remembered so I can say "it" later. Gemini is not called. **Classes.** `DeskCommand` or `CommandParser` reads the verb. `MarketDataTool` does the lookup. `FinnhubAdapter` is the usual adaptee.

Methods. `ResearchGateway.execute`, `MarketDataTool.execute`, `ProviderRouter.getQuote`.

Patterns. Command, Facade, Composite, Adapter.

Kind. Deterministic.

Video. Not public.

2.5 UC-05 Get a security overview

Stage 1. UC-05. Modified the same way as the quote.

`overview NVDA` follows the quote path, but it calls `getSecurityOverview`. Finnhub's free profile gives me the name, exchange, currency, and an industry label. It does not give a company description, and the screen says that. An ETF often has no profile at all. Deterministic. **Classes.** Same gateway and `MarketDataTool`.

Methods. `ProviderRouter.getSecurityOverview`.

Patterns. Command, Facade, Composite, Adapter.

Video. Not public.

2.6 UC-06 Analyze ETF exposure

Stage 1. UC-06. Modified. Country weights are headquarters when the free plan refuses the factsheet. I do not ask Gemini to narrate the weights.

`sector QQQ` loads the ETF profile and `ExposureCalculator` reports the provider's sector weights. `geo QQQ` tries Finnhub's published country report. On HTTP 403, `HoldingCountryResolver` looks up the headquarters of at most 25 holdings. A holding with no profile stays Unknown. Weight the provider left blank stays unassigned. `geo NVDA` has nothing to break down, because a stock has no holdings list. Provider errors go through `ProviderText.redact` so a key cannot leak into the window. **Classes.** `ETFExposureTool` decides sector versus country. `AlphaVantageAdapter` supplies holdings. `FinnhubMarketDataClient` is the country source and the headquarters lookup.

Methods. `ETFExposureTool.execute`, `calculateSectorExposure`, `HoldingCountryResolver.apply`, `calculateGeographicExposure`.

Patterns. Command, Adapter, Composite.

Kind. Deterministic.

Video. Not public.

2.7 UC-07 Compare securities

Stage 1. UC-07. Modified. No model on the direct command.

`compare NVDA MSFT` loads two quotes. `SecurityComparisonCalculator` subtracts the percent changes. One symbol is not enough, and the desk asks for the second. Deterministic. **Classes.** `ComparisonTool`, `SecurityComparisonCalculator`, `ProviderRouter`.

Methods. `ComparisonTool.execute`.

Patterns. Command, Composite, Adapter.

Video. Not public.

2.8 UC-08 Analyze ETF overlap

Stage 1. UC-08. Modified. No model on the direct command.

`overlap QQQ SPY` loads both holding lists. `ExposureCalculator.calculateOverlap` reports the names they share. Deterministic. **Classes.** `ComparisonTool`, `ExposureCalculator`, `AlphaVantageAdapter` through the router.

Methods. `calculateOverlap`.

Patterns. Command, Adapter.

Video. Not public.

2.9 UC-09 Retrieve financial news

Stage 1. UC-09. Modified.

`news NVDA` asks `NewsTool`, which asks the router. Marketaux is used when its token is set. Otherwise Finnhub company news is used. I left Alpha Vantage news unregistered because that free plan is about 25 calls a day. A headline match means the ticker was attached to the article. It does not, by itself, mean the article caused a move. If I ask why the price moved, that is UC-10, not this list. Deterministic for the list itself. A spoken reply on top of the list, when Gemini is set, is hybrid and has to stick to the article text. **Classes.** `NewsTool`, `MarketauxAdapter`, `FinnhubAdapter`.

Methods. `NewsTool.execute`, `ProviderRouter.getNews`.

Patterns. Command, Composite, Adapter.

Video. Not public.

2.10 UC-10 Explain a recent move

Stage 1. UC-10. Modified. The note is written from the articles, and it has to finish.

I type `why did microsoft fall` or `why is intel up`. `nasdaq` is read as `QQQ`, and the note has to say QQQ is an ETF. `MarketMovementTool` loads the quote and up to three articles. `NewsTiming` marks anything published after the last trade. `MarketExplanationService.explain` calls `generate`, not the short call, and Flash-Lite is told to think as little as possible so the note is not cut off after the price.

The note names the symbol, the price, and the percent change. It then explains the move from what the articles say, and it attributes each reason to the reporting. An article that came out after the last trade is still summarized, but it is labeled as later context, not as proof it caused that print. If no article states a reason, the note says the evidence does not establish a cause. It does not recommend a buy or a sell. The answer is shown first. The quote and the sources follow. **Classes.** `MarketMovementTool` gathers evidence. `MarketExplanationService` writes the prompt. `GeminiLLMClient` calls the API. `ResponseComposer` lays out the screen.

Methods. `MarketMovementTool.execute`, `explain`, `LLMProvider.generate`.

Patterns. Command, Adapter, Strategy (`LLMProvider`).

Kind. Hybrid. The evidence is deterministic. The note is the model.

Video. Not public.

2.11 UC-11 Add a portfolio position

Stage 1. UC-11. Modified. This is no longer a tool the model can call.

`port add NVDA 10 STOCK` goes to `SessionWorkspace.addPosition`, then `PortfolioService`, then `SQLitePortfolioRepository`. The type is `STOCK`, `ETF`, or `OTHER`. A sentence cannot add a position. Deterministic. **Classes.** `PortfolioPanel`, `DeskCommand`, `PortfolioService`.

Methods. `addPosition`, `PortfolioRepository.saveForUser`.

Patterns. Command, Repository, Facade.

Video. Not public.

2.12 UC-12 Remove a portfolio position

Stage 1. UC-12. Modified for the same reason.

`port remove NVDA` deletes that symbol for me. If it was not there, the screen says so and leaves the rest alone. The model cannot do this. Deterministic. **Classes.** `PortfolioService`, `SQLitePortfolioRepository`.

Methods. `removePosition`.

Patterns. Command, Repository.

Video. Not public.

2.13 UC-13 View the portfolio

Stage 1. UC-13. Implemented.

`portfolio`, or the Portfolio screen, lists what is stored. No network call is required just to list the rows. Prices show up when something else quotes them. Deterministic. **Classes.** `PortfolioPanel`, `TerminalUI`, `SessionWorkspace`.

Methods. `PortfolioService.getPortfolio`, `loadForUser`.

Patterns. Repository, Facade.

Video. Not public.

2.14 UC-14 Manage the watchlist

Stage 1. UC-14. Modified. There is no `WatchlistTool` on the agent.

`watch add NVDA` and `watch remove NVDA` go through `WatchlistService` and `SQLiteWatchlistRepository`. A watchlist item has no quantity. Deterministic. **Classes.** `WatchlistPanel`, `WatchlistService`.

Methods. `addSymbol`, `removeSymbol`.

Patterns. Repository, Command.

Video. Not public.

2.15 UC-15 Analyze portfolio exposure

Stage 1. UC-15. Modified. Java does the arithmetic, and Gemini does not narrate it.

`exposure` loads my positions, quotes each one, and sets value to price times quantity. Day move is the price change times quantity. Currencies stay in separate books. A position with no price is named and left out of the total. ETF sector weights are look-through when the holding list exists. If a sentence selects this action, the numbers still are not sent to the model. **Classes.** `PortfolioAnalysisTool`, `PortfolioExposureCalculator`, `PortfolioBook` for the home summary.

Methods. `PortfolioAnalysisTool.execute`, `calculate`.

Patterns. Command, Composite, Adapter.

Kind. Deterministic.

Video. Not public.

2.16 UC-16 Ask a follow-up

Stage 1. UC-16. Modified. The model may only name an allowed action, and a rephrase does not run a tool.

I type in the message box, for example `what about its sector exposure`, or at the terminal with no slash. The window shows my line, then `ResearchGateway.ask` calls `AgentController.handle`. `ConversationMemory` reads my recent turns and the last symbol. `RequestPlanner` always builds a keyword plan first. If Gemini is configured, it may replace that plan with one allowed JSON action. A bad reply falls back to the keywords.

If the plan is a lookup, `ToolManager` runs it. A move plan then explains the move from the articles. If I say "say that more simply", the plan is `DISCUSS`: nothing is fetched, and Gemini rewrites the stored exchange. Portfolio figures are left out of that prompt. Both sides of the exchange are stored for me only. Another user's rows are never read.

A sentence that says to ignore previous instructions, or that looks like SQL, is rejected and does not run a tool. Compare and overlap still need two symbols. **Classes.** `AgentPanel` is the conversation. `TerminalUI` is the other door. `RequestPlanner`, `PromptBuilder`, `ConversationMemory`, `ResponseComposer`.

Methods. `AgentController.handle`, `createPlan`, `remember`, `lastSymbol`.

Patterns. Command, Strategy, Facade, Repository.

Kind. Hybrid when Gemini is set. Keyword-only when it is not.

Video. Not public.

2.17 UC-17 Read financial statements

Stage 1. New.

`statements NVDA` goes through `MarketInsightTool` to `FinancialModelingPrepAdapter`. I show up to four annual periods. A line the filing omitted stays blank. Without `FMP_API_KEY` the command says it is unavailable. Deterministic. **Classes.** `MarketInsightTool`, `FinancialModelingPrepAdapter`, `ProviderRouter`.

Methods. `getFinancialStatements`.

Patterns. Command, Adapter, Composite.

Video. Not public.

2.18 UC-18 Read price history

Stage 1. New.

`history NVDA` asks Twelve Data for recent daily bars and the latest 14-day RSI. Quotes still work from Finnhub if this key is missing. Deterministic. **Classes.** `MarketInsightTool`, `TwelveDataAdapter`.

Methods. `getPriceHistory`.

Patterns. Command, Adapter.

Video. Not public.

2.19 UC-19 Read sentiment

Stage 1. New.

`sentiment NVDA` asks SentiSense for the latest score, from -1 to 1, and shows my own band label for that score. The band is a label, not advice. I did not buy the paid SentiSense plan. Deterministic. **Classes.** `MarketInsightTool`, `SentiSenseAdapter`, `SentimentBands`.

Methods. `getSentiment`.

Patterns. Command, Adapter.

Video. Not public.

2.20 UC-20 View the home desk

Stage 1. New. Window only.

After sign-on, F1 shows market value, the day move, one row per position, and a tape of index quotes, portfolio quotes, and a few headlines. `DashboardPanel` asks `PortfolioBook` for the figures

and `TapeLine` for the tape. Network calls run off the Swing thread. The terminal has no tape. The same positions are on `portfolio`. Deterministic. **Classes.** `DashboardPanel`, `PortfolioBook`, `TapeLine`, `BackgroundWork`.

Methods. The panel's book load, `TapeLine.pieces`.

Patterns. Facade. The panel calls the gateway rather than a vendor.

Video. Not public.

3 Implemented design patterns

I am counting five. Adapter, Composite, Command, Facade, and Factory. Strategy and Repository are also in the code, and I mention them where they matter. I did not get an extra pattern by using the same idea twice.

3.1 Adapter

The vendors do not return the same JSON. I wanted the tools to ask for a quote without knowing which site they were calling. `DataProvider` is the shape they share. `FinnhubAdapter`, `AlphaVantageAdapter`, `MarketauxAdapter`, `FinancialModelingPrepAdapter`, `TwelveDataAdapter`, and `SentiSenseAdapter` are the adapters. `FinnhubMarketDataClient` and `AlphaVantageClient` are the older clients sitting behind two of them. The files are under `src/main/java/com/marketintel/providers/`.

Stage 1 had one client class per provider interface. That was already a thin adapter, but it assumed one vendor. Once I stayed on free plans, I needed several. Putting an adapter in front of each vendor is what let me swap Finnhub in for quotes without rewriting `MarketDataTool`.

3.2 Composite

The tools still want one object. `ProviderRouter` is that object. It implements `MarketDataProvider`, `NewsProvider`, `ETFDataProvider`, and the statement, history, and sentiment interfaces. `prefer` registers an adapter for one capability. The first one registered handles that capability. A later one is used only if the earlier one was never registered. It is not a failover chain. Stage 1 had no router. One client was the provider.

3.3 Command

The window, the terminal, and the planner all need to run the same research actions. `ActionType` is the name. `AgentTool` is the command, with `supports` and `execute`. `ToolManager` finds the tool and runs it. `DeskCommand` and `CommandParser` turn what I typed into that name. Statements, history, and sentiment share one tool, `MarketInsightTool`, instead of three copies of the same plumbing. Portfolio edits are commands too, but they stop at `SessionWorkspace`. The planner is not allowed to issue them.

Stage 1 already had `AgentTool`. I kept it. What changed is that the model cannot use it to edit the portfolio.

3.4 Facade

I did not want the window and the terminal to grow their own copies of the session check. `ResearchGateway` is the front door. `execute` runs one tool. `ask` handles a sentence. `explainMovement` is the direct move path. `SessionWorkspace` is the front door for login and for portfolio edits. Stage 1's terminal talked to the controller directly, and there was no window. The facade is what made the second interface safe to add.

3.5 Factory

Which adapters exist depends on which keys I exported. `ApplicationContext.create` and `createDefault` build the database, the repositories, the router, the tools, and the Gemini client. An unset key means that adapter is never constructed. Tests build fakes themselves and do not go through this factory to hit the network. Stage 1 showed the objects, but not one place that decides which of them exist.

3.6 Also in the code

`PasswordHasher` with `BCryptPasswordHasher`, and `LLMProvider` with `GeminiLLMClient`, are Strategy. The keyword plan is what I use when the model is missing or its answer is rejected. The four repository interfaces each have one SQLite class, so the services do not write SQL.

3.7 Diagrams

These match the code as of 8 November. The old Stage 1 renders are still in `docs/Stage1-pdfs/`. They show an LLM call on a quote, and classes I never built, including `UserPreferences` and `ConversationSession`.

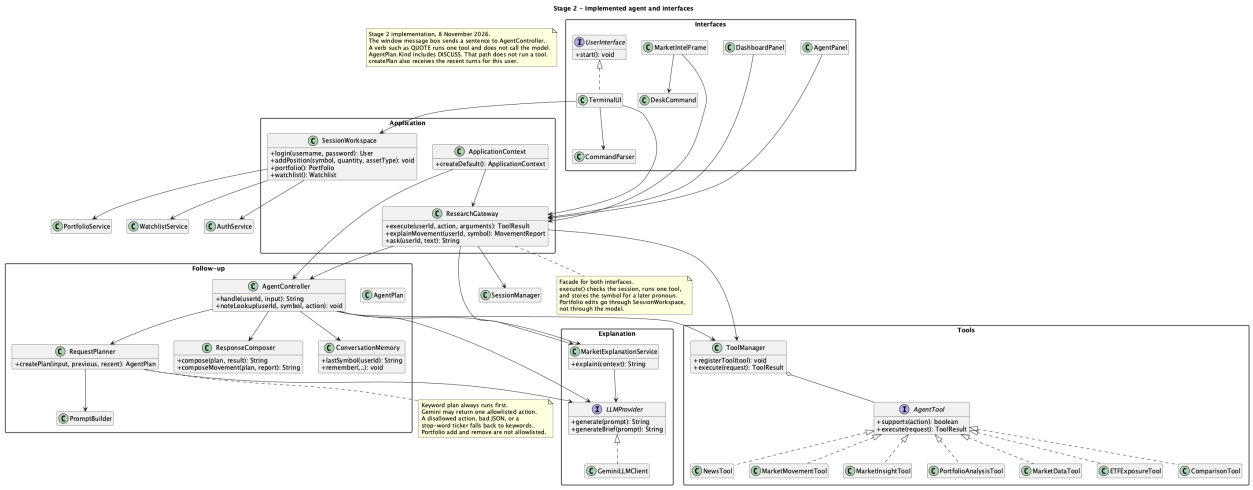


Figure 1: The window, the terminal, and the follow-up. A sentence goes through `AgentController`. A quote does not.

The other sources, for ETF analysis, country exposure, portfolio exposure, and the insight commands, are in `docs/uml/`. Rendered copies are in `docs/uml/rendered/`.

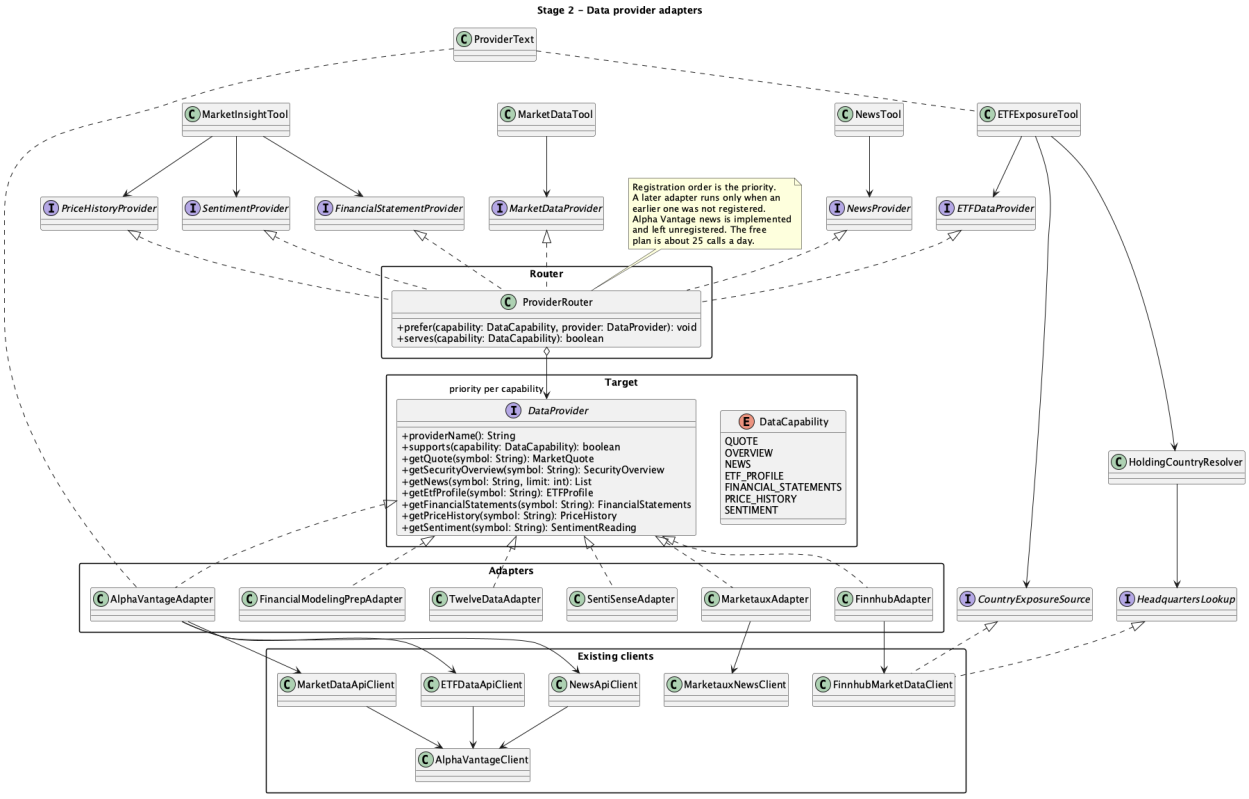


Figure 2: The adapters and `ProviderRouter`.

4 Changes and rationale

I kept the Stage 1 use cases. Where the running desk differs, this is why.

Providers. Stage 1 had one client per provider interface, and Alpha Vantage was the market source. I put six adapters behind `ProviderRouter`. Finnhub is first for quotes. Alpha Vantage is first for ETF holdings. Alpha Vantage turned down a free educational plan, and the free tier is too small to carry the whole desk. The tools still depend on the role interfaces. An unset key drops that adapter. UC-04, UC-05, and UC-09.

Country exposure. Stage 1 was the fund's published country weights, then a model explanation. I try Finnhub's country report. On HTTP 403 I map at most 25 holdings to headquarters countries, and I show the weights without calling Gemini. The free plan returns 403. Filling in the rest would have been an invented allocation. The user still gets a country breakdown, and the screen says it is headquarters, not the factsheet. UC-06.

Where Gemini sits. Stage 1 sent every research use case through the model. A verb such as `quote` runs one tool. Gemini is on the move note and on sentences. A quote does not need a model, and a model there was inventing a sentence around a number I already had. UC-04 through UC-09 and UC-15 stay deterministic. UC-10 and UC-16 are the hybrid ones.

How a move is explained. Stage 1 had the composer ask the model to explain, including when evidence was missing. I load the quote and the news first. The note has to use what the articles say, attribute the reason to the reporting, and still finish. An article after the last trade is later

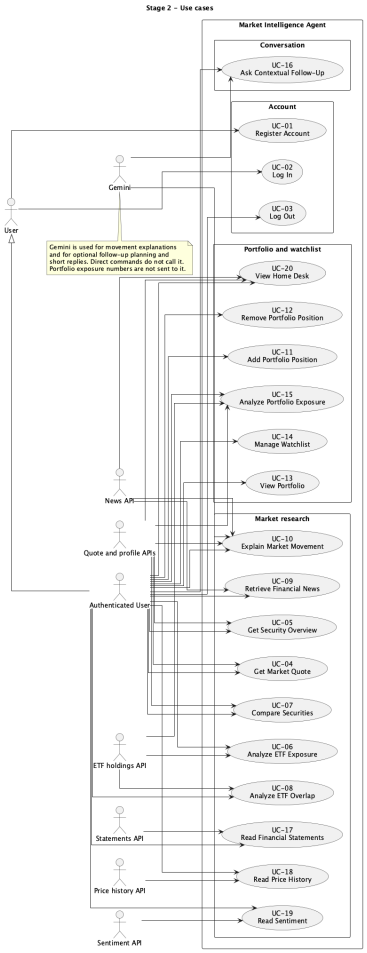


Figure 3: Use cases. Gemini is only on the move note and the follow-up.

context, not proof. The first notes were either cut off after the price, or they said the articles could not explain a move that the articles did discuss. UC-10. `MarketExplanationService`, `generate`, minimal thinking on Flash-Lite.

Portfolio exposure. Stage 1 had the model explain the exposure. Java multiplies price by quantity. Those numbers are not sent to Gemini. The figure is arithmetic. Narrating it invited a recommendation. UC-15.

Portfolio edits. Stage 1 put add and remove on the agent tools. They stay on `SessionWorkspace`, off the allowlist. A sentence must not change what I hold. UC-11, UC-12, UC-14. There is no `PortfolioTool` class.

The window. Stage 1 was a terminal. I added the Swing desk, then replaced the command line with a message box so a sentence and a verb share one place. Stage 2 asks for both a GUI and a CLI. UC-20 is new. UC-16 is how the box talks to the planner.

Statements, history, sentiment. These were not in the Stage 1 use-case set. UC-17, UC-18, and UC-19, through `MarketInsightTool`. The free adapters were already there for the provider pattern. The commands are the part a person can see. Each one is unavailable when its key is unset. They do not replace a Stage 1 feature.

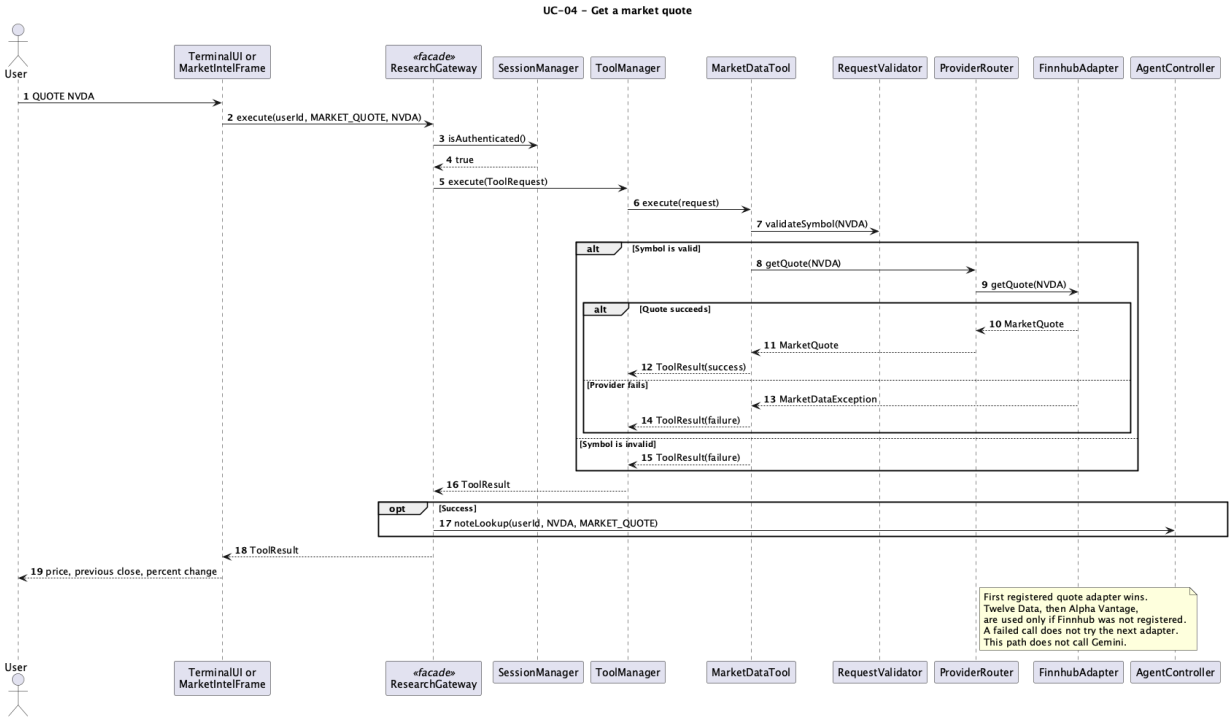


Figure 4: UC-04. A quote is a tool call.

Keys in errors. Stage 1 did not specify this. A live Alpha Vantage key showed up in a rate-limit message. `ProviderText.redact` replaces that notice. I rotated the key. It is not in this report.

5 AI tools and collaboration log

Cursor is a $20 a month subscription. Most of the work on this desk was with Grok 4.7 High Fast, and that model is included in the subscription. Over the past 30 days the Cursor account used 226.1 million tokens: 218.2 million of those were included, and 7.9 million were on-demand. The on-demand rows I pulled from the usage page come to $1.86. Claude 4.5 Sonnet is $1.59 of that, and Gemini 3.1 Pro is $0.27. Those rows are about 3.0 million tokens, so they are not the whole 7.9 million. I am not filling in a dollar amount for the rows that are not on the page. Claude Fable was also used, inside the included allowance. The Gemini calls the running desk makes, through the API, are on the free tier.

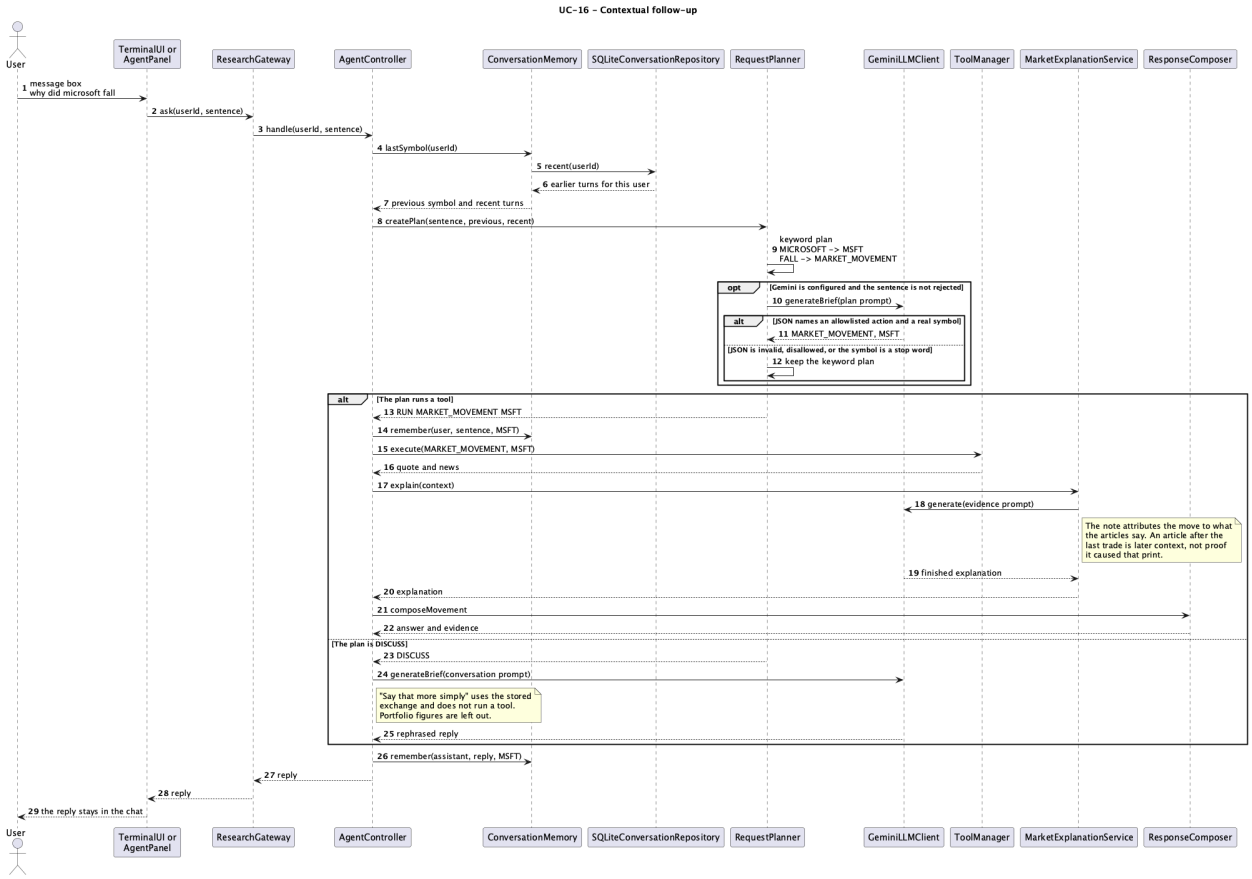


Figure 5: UC-16. A sentence is planned. A rephrase does not run a tool.

Tool and model	What I used it for	Cost
Cursor, mainly Grok 4.7 High Fast. Also Claude Fable, Claude 4.5 Sonnet, and Gemini 3.1 Pro.	Most of the Java, the tests, the diagrams, and the drafts of this report. I ran the window myself and threw out the parts that were wrong.	$20 a month. Past 30 days: 226.1M tokens, 218.2M included and 7.9M on-demand. On-demand rows on the usage page total $1.86.
ChatGPT. The model name was not written down for the early design chats.	The Stage 1 split into tools, providers, and repositories.	Not written down.
Gemini API, inside the running desk. Default `gemini-3.5-flash-lite`. I tried `gemini-3.8-flash` and `gemini-2.5-flash` and stopped using them.	The move note, and the optional planning and replies on a sentence. Not quotes, not portfolio math, not add or remove.	Free tier. No invoice.

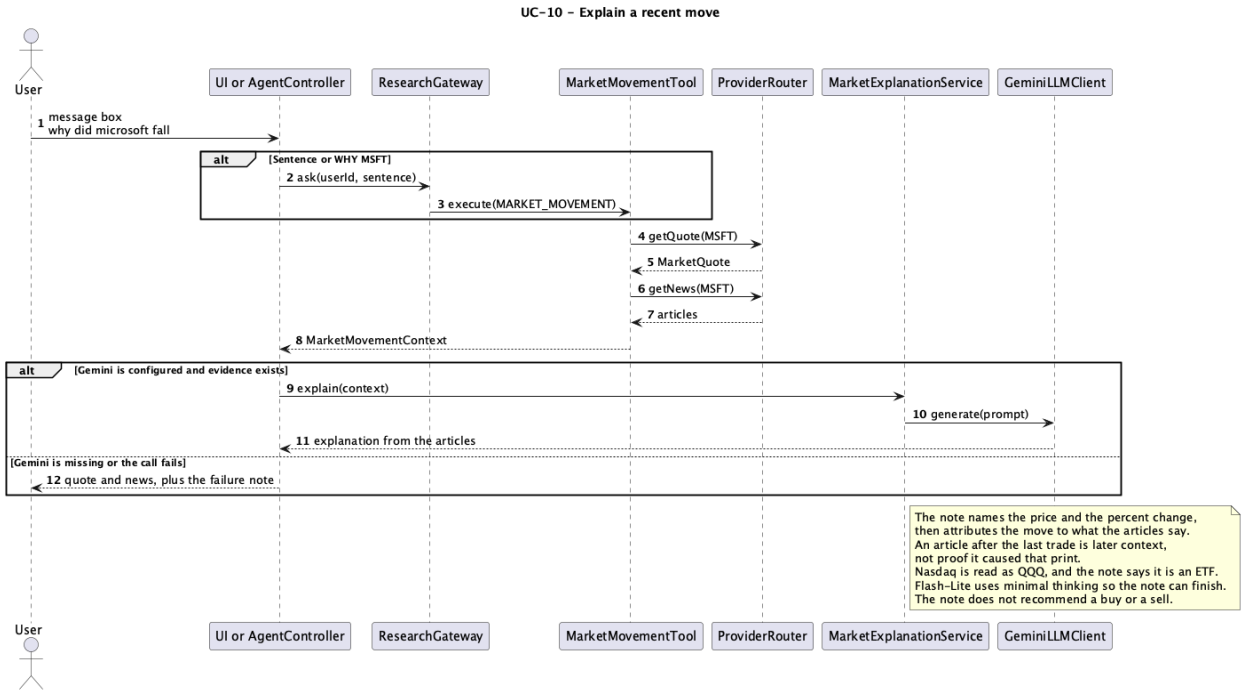


Figure 6: UC-10. Quote and news first. Gemini explains the move from the articles.

The longer notes are in `docs/ai-collaboration-log.md`. The ones that actually changed the desk:

- Early on, ChatGPT proposed the tool and provider split. I added accounts, portfolios, SQLite, and `BigDecimal`. That split is still the package layout.
- After Alpha Vantage declined a free educational plan, I asked for one adapter per vendor. That became `DataProvider` and `ProviderRouter`.
- I saw HTTP 403 on country exposure, and then I saw my Alpha Vantage key in an error. The headquarters fallback and `ProviderText.redact` came out of that. I rotated the key.
- Ordinary questions were missed, Gemini timed out, a new key got HTTP 404 from `gemini-2.5-flash`, and "why did microsoft fall" was read as the ticker DID. Memory, the allowlist, Flash-Lite, and the sentence parser came out of those runs. I checked the Microsoft question in the window.
- The message box was still cutting the note off after the price, and a news reply was refusing to use the articles as the reason. The move note now has to finish, and it has to attribute the move to what the articles say.

The rule I kept, after the model was wired in, is that Gemini may interpret a question, but the numbers come from a tool. It does not get to edit the portfolio, and a sentence that tells it to ignore the instructions does not run anything.

6 Development and verification summary

Dates are Toronto dates on the public repository. I am not counting a merge commit as the work. The work is the commit that was merged. A report-only edit does not count as one of the five.

6.1 Weekly commits

Before 20 October the repository already had about twenty commits, from the Stage 1 design through the first terminal work. I work full time, so I did not push during the week of 20–26 October. New commits start again on 27 October.

Period 2, 27 October–2 November 2026, is over the minimum. Five of them:

- 3770181 – 28 October. One wiring for the terminal and the window.
- f103405 – 28 October. The Swing window.
- fcd3877 – 29 October. Exposure from price times quantity.
- 4da99b3 – 29 October. Free APIs behind adapters.
- c75e138 – 29 October. Follow-up from the last symbol.

Also that week: `6843eed` Finnhub quotes, `9716283` country exposure without printing keys, `1f4db1c` the home tape, `04de078` Gemini reading a question, `4f52ca0` Flash-Lite, and `931ec38` so "why did microsoft fall" stays a question.

Period 3, 3–9 November 2026, also clears five. I am not borrowing Period 2 commits for it.

- d5d220c – 5 November. Home value, weight, and unpriced names.
- 1bcaa63 – 6 November. One in-flight quote shared by the tape and the home book.
- a0838f5 – 6 November. A real reason when a holding cannot be priced.
- 6eef95e – 8 November. Terminal commands without a required slash.
- f1d9eaf – 8 November. The window box became a conversation.
- 9f71542 – 8 November. The move note uses what the articles say.

`d7aa5d3` the same day lets a follow-up continue the last reply, and `1ae3345` stops a note from ending after the price.

6.2 What I checked

`./mvnw test` does not call the live APIs. `GeminiLiveSmokeTest` runs only if I set `RUN_GEMINI_LIVE_TEST=1` and a Gemini key. On 8 November the planner tests, the explanation-prompt tests, and the Gemini client tests passed after the article-explanation change.

I also used the window with real keys. That is how I found the defects below. The test suite does not replace that, and that does not replace the suite.

6.3 Defects I hit and fixed

The country URL returned HTTP 302. It now calls https://api.finnhub.io/api/v1/etf/country. HTTP 403 on that report is the free plan, not a rejected key. Quotes still work.

An Alpha Vantage error included the live key. `ProviderText.redact` strips it. I rotated the key.

`why did microsoft fall` was read as the symbol DID. A full sentence stays a question. `WHY MSFT` is still the short command.

The move note stopped mid-word after the price, because Flash-Lite was spending the token budget on thinking. Explanations now use minimal thinking and the longer call. A reply cut off by the token limit is not shown as a finished note.

A news answer said the articles did not explain the day's move, even when they discussed the 40% correction, demand, and guidance. A question about a price rising or falling stays a movement explanation, and the note has to use what the articles say.

6.4 Still open

The videos are not public yet. Country exposure on this plan is headquarters, and that is the behavior, not a placeholder. The heavier testing and the KUMA checks are Stage 3, which is what the Stage 2 instructions say.

7 Demonstration index

I do not have public links in this version. Slides are not a demonstration. Before 10 November I need a public link and a timestamp on every row. Two recordings are enough if each row is a labeled segment. I say the use-case id at the start of the segment. One recording is the window. One is the terminal. I do not show an API key, a `.env` file, or a shell line that prints a key.

The window box is the message field at the bottom. Enter sends. The terminal is `./mvnw compile exec:java -Dexec.args="-cli"`. No slash.

Feature	What I need on screen	Link
UC-01 Register	Create-account screen. Username, password, and the same password again. Then the return to sign-on. I am not signed in yet.	Not public
UC-02 Log in	Sign-on has no confirmation field. A wrong password, then a real sign-on, then the home desk.	Not public
UC-03 Log out	Window logout back to sign-on. Terminal `logout` back to the menu.	Not public
UC-04 Quote	`quote NVDA` in the window and in the terminal. Price, previous close, percent change.	Not public
UC-05 Overview	`overview NVDA`. Name, exchange, currency, industry. I say the free profile has no description.	Not public
UC-06 Exposure	`sector QQQ`, then `geo QQQ`, then `geo NVDA`. Sector weights, headquarters countries with the factsheet note, and no country list for the stock.	Not public
UC-07 Compare	`compare NVDA MSFT`. Both changes and the difference.	Not public
UC-08 Overlap	`overlap QQQ SPY`. At least one shared holding.	Not public
UC-09 News	`news NVDA`. Headlines. I say a match is not, by itself, the cause.	Not public
UC-10 Move	`why did microsoft fall`, then `why is intel up`. The note names the price and uses the articles. A later headline is called later context. The not-advice line is visible.	Not public
UC-11 Add	`port add NVDA 10 STOCK`. The stored row.	Not public
UC-12 Remove	`port remove NVDA`, then a symbol I do not hold. The missing-symbol message.	Not public
UC-13 View	Terminal `portfolio` and the Portfolio screen. Same rows.	Not public
UC-14 Watchlist	`watch add NVDA`, then `watch remove NVDA`. The list after each.	Not public
UC-15 Exposure	`exposure`. Value is price times quantity. An unpriced name stays out of the total.	Not public
UC-16 Follow-up	After UC-10: `what about its sector exposure`, then `say that more simply`, then a sentence that says to ignore previous instructions. The old symbol is reused. The rephrase does not look up a new one. The injection does not run a tool.	Not public
UC-17 Statements	`statements NVDA`, or the unavailable message if the key is unset.	Not public
UC-18 History	`history NVDA`. Bars and RSI.	Not public
UC-19 Sentiment	`sentiment NVDA`. Score and band. I say the band is not advice.	Not public
UC-20 Home	F1 after sign-on. Value, day move, position rows, and the tape. Window only.	Not public

The terminal segment has to include `quote`, `why did microsoft fall`, and `exposure`. The window segment has to include sign-on, home, the message box, and at least one research sentence. That is both interfaces.